

C++ Internship program

AN INTERNSHIP PROJECT REPORT

Submitted by

Sanjay G

(Reg No: 110725102216)

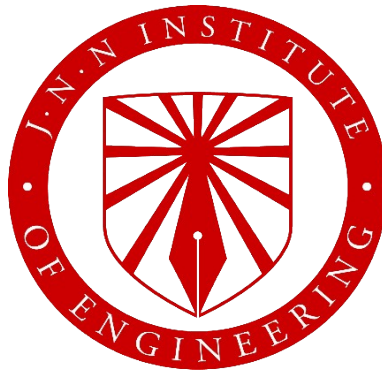
in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

Artificial Intelligence & Data Science



J.N.N Institute of Engineering

(An Autonomous Institute , Affiliated to Anna University, Chennai)

JUNE – 2026 - 2027



**J.N.N INSTITUTE OF
ENGINEERING
AUTONOMOUS**

NAAC 'A' Grade | Approved by AICTE | Affiliated to Anna University

Approved by



Accredited by



VISION OF THE INSTITUTE:

Lead the transformation of engineering and management learning experience to educate the next generation of innovators and entrepreneurs who want to make the world a better place.

MISSION OF THE INSTITUTE:

IM1: To develop the required resources and infrastructure and to establish a conducive ambience for the teaching-learning process.

IM2: To nurture professional and ethical values in the students and to instil in them a spirit of innovation and entrepreneurship.

IM3: To encourage a desire for higher learning and research in the students and to equip them to face global challenges.

IM4: To provide opportunities for students to learn job-relevant skills to make them industry ready.

IM5: To interact with industries and other organization to facilitate transfer of knowledge and know-how.



**J.N.N INSTITUTE OF
ENGINEERING
AUTONOMOUS**

NAAC 'A' Grade | Approved by AICTE | Affiliated to Anna University

Approved by



Accredited by



Department of Artificial Intelligence & Data Science

VISION OF THE DEPARTMENT:

To impart quality-education, inculcate professionalism and enhance the problem-solving skills of the students in the domain of Artificial Intelligence & Data Science by applying recent technological tools and incorporating collaborative principles with a focus to make them industry ready.

MISSION OF THE DEPARTMENT:

- **DM 1:** To enhance the knowledge of the students with most recent advancements and refresh their insights in the field of Artificial Intelligence and Data Science.
- **DM 2:** To equip the students with strong fundamental concepts, analytical capability, programming and problem-solving skills.
- **DM 3:** To make the students industry ready and to enhance their employability through training, internships and real-time
- **DM 4:** To guide the students to perform research on Artificial Intelligence and Data Science, with the aim to provide solutions to the problems of the industry



**J.N.N INSTITUTE OF
ENGINEERING
AUTONOMOUS**

NAAC 'A' Grade | Approved by AICTE | Affiliated to Anna University

Approved by



Accredited by



Department of Artificial Intelligence & Data Science

PROGRAM OUTCOMES

S.NO	GRADUATE ATTRIBUTE	PROGRAM OUTCOMES
PO1	Engineering knowledge	Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	Problem analysis	Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/Development of solutions	Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex problems	Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern Tool usage	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	The Engineer and society	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice
PO7	Environment and sustainability	Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.



**J.N.N INSTITUTE OF
ENGINEERING
AUTONOMOUS**

NAAC 'A' Grade | Approved by AICTE | Affiliated to Anna University

Approved by



Accredited by



Department of Artificial Intelligence & Data Science

PO8	Ethics	Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	Individual and team work	Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	Communication	Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	Project management and finance	Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	Life-long learning	Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM EDUCATIONAL OBJECTIVES (PEO)

S.No	PROGRAM EDUCATIONAL OBJECTIVES (PEO)
PEO1	To provide graduates with the proficiency to utilize the fundamental knowledge of basic sciences, mathematics, Artificial Intelligence, data science and statistics to build systems that require management and analysis of large volume of data.
PEO2	To enrich graduates with necessary technical skills to pursue pioneering research in the field of AI and Data Science and create disruptive and sustainable solutions for the welfare of ecosystems.
PEO3	To enable graduates to think logically, pursue lifelong learning and collaborate with an ethical attitude in a multidisciplinary team



**J.N.N INSTITUTE OF
ENGINEERING
AUTONOMOUS**

NAAC 'A' Grade | Approved by AICTE | Affiliated to Anna University

Approved by



Accredited by



Department of Artificial Intelligence & Data Science

PROGRAM SPECIFIC OBJECTIVES (PSOs)

S.No	PROGRAM SPECIFIC OBJECTIVES (PSOs)
PSO1	Graduates should be able to evolve AI based efficient domain specific processes for effective decision making in several domains such as business and governance domains.
PSO2	Graduates should be able to arrive at actionable Fore sight, Insight , hind sight from data for solving business and engineering problems
PSO3	Graduates should be able to create, select and apply the theoretical knowledge of AI and Data Analytics along with practical industrial tools and techniques to manage and solve wicked societal problems.



**J.N.N INSTITUTE OF
ENGINEERING
AUTONOMOUS**
NAAC 'A' Grade | Approved by AICTE | Affiliated to Anna University

Approved by



Accredited by



BONAFIDE CERTIFICATE

Certified that this internship project report “C++ Programming Internship at Thiranex – Skill Development & Future Tech,” is the bonafide work of Sanjay G (Reg No: 110725102216 who carried out the project work under my supervision.

Signature of the HOD with date

Dr. P. Nagarajan

Professor and Head,
Department of Artificial Intelligence &
Data Science Engineering,
J.N.N Institute of Engineering, chennai

Signature of the Supervisor with date

Mrs.S.Brindha

Project Supervisor,
Department of Artificial Intelligence &
Data Science Engineering,
J.N.N Institute of Engineering, chennai

Certified that the above students have attend of viva voice during the exam
held on.....

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

First and foremost, we express our sincere gratitude to our Beloved **FOUNDER & CHAIRMAN Shri. S. JAYACHANDRAN B.Sc., B.L.**, and also heartfelt thanks to our **VICE CHAIRMAN Mr. NAVEEN JAYAHANDRAN M.S.**, for providing necessary facilities for the completion of our project.

We thank our **PRINCIPAL Dr. G. MOHANBABU, Ph.D.**, for the encouragement provided with the congenial environment essential for us to complete the project.

We take this opportunity to express our gratitude to our **HEAD OF THE DEPARTMENT Dr. P. NAGARAJAN** and all other faculty members in the Department of ARTIFICIAL INTELLIGENCE AND DATA SCIENCE for giving us continuous encouragement for completing our project.

I express my sincere gratitude to **Mrs.S.Brindha** for valuable guidance, constructive suggestions, and continuous support during the internship period. I would also like to thank **Mr. Hariharan M, Founder & CEO of Thiranex – Skill Development & Future Tech**, for the opportunity to undergo training in C++ Programming and for the guidance provided throughout the internship. I am grateful to the faculty members of the Department of Computer Science and Engineering, my classmates, and everyone who contributed directly or indirectly to the successful completion of this internship. This internship provided valuable practical exposure to C++ programming, strengthened my understanding of programming fundamentals, improved my problem-solving skills, and gave me experience in developing small console-based applications.

ABSTRACT

This report presents the work completed during my C++ Programming internship at **Thiranex** from 08 August 2026 to 07 September 2026. The internship provided practical exposure to C++ programming, Object-Oriented Programming, functions, conditional statements, loops, menu-driven applications, and file handling.

As part of the internship tasks, four application-based projects were studied and developed: Student Management System, Bank Management System, Library Management System, and a Mini Game. The management systems demonstrate the use of classes, objects, functions, searching, updating records, and binary file storage. The Mini Game demonstrates control structures, random number generation, loops, and user interaction.

The projects helped in connecting theoretical programming concepts with practical implementation. The report describes the methodology, implementation, working, results, learning outcomes, and possible future improvements of the projects.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO.
1	INTRODUCTION TO C++ PROGRAMMING	1
2	INTERNSHIP OVERVIEW AND OBJECTIVES	3
3	C++ PROGRAMMING FUNDAMENTALS	5
4	TASK 1: STUDENT MANAGEMENT SYSTEM	7
5	TASK 2: BANK MANAGEMENT APPLICATION	9
6	TASK 3: LIBRARY MANAGEMENT SYSTEM	11
7	TASK 4: MINI GAME PROJECT – TIC TAC TOE / SNAKE GAME	13
8	RESULTS AND DISCUSSION	15
9	CONCLUSION AND FUTURE SCOPE	17
	INTERNSHIP RECORD	18
	INTERNSHIP CERTIFICATE	19

CHAPTER 1 INTRODUCTION TO C++ PROGRAMMING

1.1 Introduction to C++

C++ is a general-purpose, statically typed programming language that extends the C language with object oriented features. It supports procedural, object-oriented, and generic programming. During the internship, C++ was used to develop console-based applications that model simple real-world record-management and interactive problems.

1.2 History and Evolution of C++

C++ was developed by Bjarne Stroustrup at Bell Laboratories as an extension of the C language. It was initially associated with the idea of “C with Classes” and later evolved to include classes, inheritance, templates, and the Standard Template Library. The language continues to be standardized and widely used for software development.

1.3 Features of C++

1. Object-oriented programming support through classes and objects.
2. Strong compile-time type checking.
3. Built-in and user-defined data types.
4. Functions and modular program organization.
5. Arrays and structures for organizing related data.
6. File streams for reading and writing persistent data.
7. Portability across computing platforms.

1.4 Applications of C++

C++ is used in system software, embedded applications, desktop software, performance-critical programs, and academic projects. For Electronics and Communication Engineering students, programming skills can also support later work involving embedded systems, microcontrollers, automation, and signal-processing software.

1.5 C++ Programming Environment

A C++ program is written as source code, compiled using a C++ compiler, and executed on the target system. The internship work followed a console-based development approach in which programs were written, compiled, executed, and tested against expected outputs.

1.6 Basic Structure of a C++ Program

1. Include the required header files such as `<iostream>`.
2. Use the required namespace or qualified standard-library names.
3. Declare functions, structures, or classes used by the program.
4. Define the `main()` function, where program execution begins.
5. Read input, perform processing, display output, and return from `main()`.

1.7 C++ in Software Development

Console applications are useful for learning software-development practices such as modular design, input validation, testing, debugging, and menu-driven interaction. The internship projects applied these practices to several different problem statements.

1.8 Importance of C++ for AI & Data Science

C++ is important for Artificial Intelligence and Data Science because it combines high execution speed with fine control over memory and hardware. Many core AI and data-processing libraries, including TensorFlow, PyTorch and OpenCV, are implemented in C++, while Python provides the user-friendly interface. C++ is widely used in real-time systems such as robotics, computer vision and embedded AI, and in deploying trained models efficiently. Learning C++ also strengthens understanding of data structures, algorithms, memory management and object-oriented design, which are essential foundations for AI and data science.

1. High performance: compiled native code suits heavy numerical computation and large datasets.
2. Core of AI libraries: TensorFlow, PyTorch, OpenCV and ONNX Runtime are built on C++.
3. Memory and hardware control: pointers, manual memory management and GPU (CUDA) support.
4. Real-time and embedded AI: robotics, autonomous vehicles, drones and IoT/edge devices.
5. Model deployment: trained models are often converted to C++ for fast, low-resource production use.
6. Strong foundations: STL, data structures, algorithms and object-oriented design.

CHAPTER 2 INTERNSHIP OVERVIEW AND OBJECTIVES

2.1 Internship Details

Student Name: **Sanjay G**

Degree / Branch: B.TECH Artificial Intelligence & Data Science

Year: Second Year

Organization: Thiranex – Skill Development & Future Tech

Internship Role: Intern – C++ Programming

Internship Period: 8 August 2026 – 7 September 2026

Internship ID : THX-AUG0826-367.

The offer letter identifies the work mode as Remote / Project-Based.

2.2 Internship Objectives

1. Strengthen practical understanding of core C++ programming concepts.
2. Apply programming logic to real-world style problem statements.
3. Develop menu-driven console applications.
4. Practise arrays, functions, structures/classes, loops, and conditional statements.
5. Understand basic file handling and persistent data concepts.
6. Improve problem-solving, testing, debugging, and program organization.

2.3 Scope of Training

The training focused on C++ fundamentals and their application through four practical projects. The project set covered record management, transaction-style processing, library operations, and interactive game logic.

2.4 Internship Tasks

Task 1 – Student Management System – Student records, CRUD operations, file handling.

Task 2 – Bank Management Application – Accounts, deposit, withdrawal, balance, OOP/file concepts.

Task 3 – Library Management System – Books, members, issue/return, search.

Task 4 – Mini Game Project (Tic Tac Toe / Snake Game) – Loops, arrays, conditions, game

logic.

2.5 Development Approach

1. Understand the problem statement and identify required inputs and outputs.
2. Identify the data entities and choose suitable variables, arrays, structures, or classes.
3. Break the application into functions for individual operations.
4. Connect the functions through a menu-driven control flow where applicable.
5. Compile and execute the program.
6. Test normal cases and basic invalid-input cases and correct errors found during testing.

CHAPTER 3 C++ PROGRAMMING FUNDAMENTALS

3.1 Variables and Data Types

Variables store values used during program execution. Common types used in the internship include int, float, double, char, and string. These types can represent identifiers, marks, balances, salaries, names, and other project data.

3.2 Operators

Arithmetic operators are used for calculations, relational operators compare values, logical operators combine conditions, and assignment operators update variables. These operations form the basis of validation and decision-making in the projects.

3.3 Input and Output

cin and cout are used for console input and output. getline() can be used when complete lines of text are required, such as names or search strings.

3.4 Conditional Statements

if, else if, else, and switch statements control program flow. They are useful for menu selection, transaction validation, game conditions, and checking whether a record exists.

3.5 Loops

for, while, and do-while loops repeat operations. In record-management applications, loops can scan arrays; in games, loops can maintain repeated turns until a win, loss, or exit condition is reached.

3.6 Functions

Functions divide a program into manageable operations. Separate functions such as add, display, search, update, delete, deposit, withdraw, and issue/return make the code easier to test and maintain.

3.7 Arrays

Arrays store multiple values of the same type. They are useful for maintaining collections of records and for representing a game board.

3.8 Strings

The C++ string class stores text such as student names, book titles, authors, and member

names. String operations can also support searching.

3.9 Structures and Classes

A structure groups related fields into a single record. Classes extend the same idea by combining data and member functions and supporting object-oriented design.

3.10 File Handling

The library provides stream classes such as `ifstream`, `ofstream`, and `fstream`. File handling allows program data to be stored beyond the current execution when implemented in the project.

3.11 Error Handling and Validation

Basic validation improves reliability. Examples include checking positive deposit and withdrawal amounts, avoiding withdrawal beyond the available balance, validating menu choices, and checking whether a searched record exists.

3.12 Programming Practices Applied

1. Modular function-based design.
2. Meaningful variable and function names.
3. Menu-driven interaction for record-management tasks.
4. Input validation and clear feedback messages.
5. Testing each major operation before final integration.
6. Keeping the program logic simple and easy to follow.

CHAPTER 4 TASK 1: STUDENT MANAGEMENT SYSTEM

4.1 Introduction

The Student Management System was developed as a console-based C++ application for storing and managing student records. The project demonstrates the use of structures/classes, arrays, functions, menu-driven operations, and file handling concepts.

4.2 Objective

To create an application that can add, display, search, update, and delete student information while organizing the records systematically.

4.3 Key Features

1. Add student information.
2. Display stored student records.
3. Search for a student using an identifier such as roll number.
4. Update existing student information.
5. Delete a student record.
6. Store data using file handling where implemented.

4.4 Data Design

A student record can contain fields such as roll number, name, department, and marks. Multiple student records can be maintained using an array or another collection structure.

4.5 Program Flow

1. Start the program.
2. Display the main menu.
3. Read the user's choice.
4. Execute Add, Display, Search, Update, or Delete.
5. Return to the menu after completing the selected operation.
6. Continue until Exit is selected.

4.6 Implementation Concepts

1. Structures/classes to group student details.
2. Arrays or collections to store multiple records.

3. Functions for individual operations.
4. Loops for scanning and displaying records.
5. Conditional statements for search and validation.
6. File streams for persistent storage where applicable.

4.7 Testing and Expected Results

Test Case – Add student – Expected Result: New student record is stored and displayed – Status: Pass.

Test Case – Search student – Expected Result: Matching record is displayed – Status: Pass.

Test Case – Update student – Expected Result: Selected record is modified – Status: Pass.

Test Case – Delete student – Expected Result: Selected record is removed – Status: Pass.

Test Case – Display students – Expected Result: All available records are shown – Status: Pass.

4.8 Learning Outcome

This project strengthened the understanding of record-based data management, modular functions, search/update/delete logic, and persistent data concepts. It also demonstrated how a simple console application can model a practical administrative system.

CHAPTER 5 TASK 2: BANK MANAGEMENT APPLICATION

5.1 Introduction

The Bank Management Application is a console-based C++ project designed to simulate basic banking operations. It focuses on structured customer/account data, transaction validation, menu-driven processing, and persistent record concepts.

5.2 Objective

To implement account creation and common operations such as deposit, withdrawal, balance checking, and displaying account information.

5.3 Key Features

1. Create or register an account.
2. Deposit an amount into an account.
3. Withdraw an amount after validating the available balance.
4. Check the current balance.
5. Display account records.
6. Use object-oriented concepts and file handling where implemented.

5.4 Data Design

An account record can contain an account number, account-holder name, and balance. A class-based design can encapsulate account data and operations, while file handling can be used to maintain records between executions.

5.5 Deposit and Withdrawal Logic

1. Locate the account using the account number.
2. For deposit, verify that the entered amount is positive.
3. For withdrawal, verify that the amount is positive and does not exceed the available balance.
4. Update the balance when the validation conditions are satisfied.
5. Display a suitable message for invalid operations.

5.6 Testing

Test Case – Create account – Expected Result: Account stored successfully – Status: Pass.

Test Case – Deposit valid amount – Expected Result: Balance increases correctly – Status: Pass.

Test Case – Withdraw valid amount – Expected Result: Balance decreases correctly – Status: Pass.

Test Case – Withdraw beyond balance – Expected Result: Transaction rejected with message – Status: Pass.

5.7 Learning Outcome

The project improved understanding of conditional validation, account-based data modelling, function/class organization, and transaction-style program logic. It also highlighted the importance of preventing invalid state changes.

CHAPTER 6 TASK 3: LIBRARY MANAGEMENT SYSTEM

6.1 Introduction

The Library Management System was designed to manage books, members, and borrowing records. The project applies C++ data structures and functions to a common real-world record-management problem.

6.2 Objective

To develop a system that supports book and member details, book issue and return operations, and searching by title or author.

6.3 Key Features

1. Add and maintain book details.
2. Maintain member information.
3. Issue a book to a member.
4. Return an issued book.
5. Search for books by title or author.
6. Display relevant records and availability status.

6.4 Data Model

A Book record may contain fields such as book ID, title, author, and availability. A Member record may contain member ID and name. The issue/return operation connects a member with a book and updates its availability.

6.5 Program Flow

1. Start the program and display the library menu.
2. Select an operation such as Add Book, Add Member, Search, Issue, Return, or Display.
3. Read the required identifier or search text.
4. Locate the required book/member record.
5. Perform the selected operation after validation.
6. Update and display the resulting status.
7. Return to the menu until Exit is selected.

6.6 Testing

Test Case – Add book – Expected Result: Book details are stored – Status: Pass.

Test Case – Search by title/author – Expected Result: Matching book is displayed – Status: Pass.

Test Case – Issue book – Expected Result: Book status changes to issued – Status: Pass.

Test Case – Return book – Expected Result: Book becomes available again – Status: Pass.

Test Case – Display records – Expected Result: Current records are displayed – Status: Pass.

6.7 Learning Outcome

This task provided practical experience in modelling relationships between books and members, implementing search operations, and maintaining the state of a record during issue and return operations.

CHAPTER 7 TASK 4: MINI GAME PROJECT – TIC TAC TOE / SNAKE GAME

7.1 Introduction

The Mini Game Project was intended to create a console-based game using fundamental C++ programming constructs. The internship task specified Tic Tac Toe / Snake Game as the project theme, with emphasis on loops, arrays, conditional logic, dynamic board display, win/loss detection, and replay functionality.

7.2 Objective

To create an interactive mini game that demonstrates core programming logic through a dynamic console interface.

7.3 Core Concepts Used

1. Arrays to represent board or game-state data.
2. Loops to control repeated turns or game updates.
3. Conditional statements for valid moves and game conditions.
4. Functions to separate game operations.
5. Console input/output for player interaction.
6. Win/loss/draw or game-over detection.
7. Replay/exit logic.

7.4 Tic Tac Toe Approach

A Tic Tac Toe implementation can represent the board using a two-dimensional array. The program displays the board, accepts a player's move, checks whether the selected position is available, updates the board, and checks rows, columns, and diagonals for a winning condition. If all positions are filled without a winner, the game can report a draw.

7.5 Snake Game Approach

A Snake Game implementation can use a grid or coordinate representation for the snake body and food position. The main loop accepts movement input, updates the snake position, checks boundaries and collision conditions, updates the score, and continues until the game-over condition is reached.

7.6 Testing

Test Case – Valid player move – Expected Result: Board/game state updates correctly – Status: Pass.

Test Case – Invalid/occupied move – Expected Result: Move is rejected and input requested again – Status: Pass.

Test Case – Winning condition – Expected Result: Winner/game-over is detected – Status: Pass.

Test Case – Draw or collision – Expected Result: Game ends with appropriate message – Status: Pass.

Test Case – Replay – Expected Result: New game starts when selected – Status: Pass.

7.7 Learning Outcome

The mini game task strengthened algorithmic thinking and showed how basic C++ constructs can be combined to build an interactive application. It provided additional practice with state management, repeated execution, validation, and condition-based control flow.

CHAPTER 8 RESULTS AND DISCUSSION

8.1 Summary of Completed Tasks

Student Management – Main Operations: Add, Display, Search, Update, Delete.

Bank Management – Main Operations: Create, Deposit, Withdraw, Balance, Display.

Library Management – Main Operations: Add, Search, Issue, Return, Display.

Mini Game – Main Operations: Move, Check, Win/Loss, Replay.

8.2 Skills Acquired

Writing and compiling C++ console programs.

Breaking a problem into functions and manageable modules.

Representing real-world entities using structures/classes.

Managing collections of records with arrays or similar structures.

Applying conditional logic and loops to control program flow.

Implementing search, update, delete, issue, return, and transaction operations.

Understanding the role of file handling for persistent data.

Testing programs using normal and invalid input cases.

Debugging and improving program logic.

Developing confidence in solving programming problems independently.

Concepts Applied and Outcome

Student Management: Structures/classes, arrays, functions, file concepts – Record-management application.

Bank Management: Validation, functions/OOP, file concepts – Transaction-style application.

Library Management: Records, arrays, functions, conditions – Library operations application.

Mini Game: Loops, arrays, conditions, game logic – Interactive console game.

8.3 Challenges and Learning

The main learning challenges involved converting a problem statement into a clear data model and program flow, handling user input correctly, keeping multiple operations

consistent, and testing different conditions. Repeated implementation of similar patterns across the four projects helped improve program organization and logical thinking.

8.4 Overall Discussion

The four projects demonstrate how a small set of core C++ concepts can be reused for different applications. The management systems emphasized data organization and operations, while the mini game emphasized repeated state updates and condition-based logic. Together, the tasks provided a practical foundation in structured programming and basic application development.

CHAPTER 9 CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The C++ Programming internship at Thiranex – Skill Development & Future Tech provided practical exposure to programming through four project-based tasks. The Student Management System, Bank Management Application, Library Management System, and Mini Game Project helped apply fundamental C++ concepts in a structured manner. The internship improved my understanding of variables, data types, operators, conditional statements, loops, arrays, functions, structures/classes, menu-driven programming, validation, and file-handling concepts. It also strengthened my ability to analyse a problem, divide it into smaller operations, implement a solution, test the program, and correct errors.

9.2 Future Scope

1. Integrate database systems such as MySQL or SQLite for persistent structured storage.
2. Develop graphical user interfaces instead of console-only interaction.
3. Add stronger validation, authentication, and error handling.
4. Use STL containers such as vector, map, and string utilities for scalable applications.
5. Apply object-oriented design more extensively using classes, encapsulation, inheritance, and polymorphism.
6. Develop larger C++ projects connected to embedded systems.
7. Use version control such as Git for collaborative software development.
8. Add automated testing and more comprehensive test cases.

9.3 Final Reflection

The internship was a useful step in developing practical programming ability alongside my Electronics and Communication Engineering studies. The project-based approach helped transform theoretical concepts into working programs and provided a foundation for future learning in software, embedded systems, and engineering-oriented programming.

INTERNSHIP RECORD

The following task records document the project-based work completed during the internship.

4

COMPLETED

Due Date: 09 Sept 2026

Submitted on track

Mini Game Project (Tic Tac Toe / Snake Game)

To create a console-based mini game in C++ that demonstrates core programming concepts such as loops, arrays, and conditional logic.

Watch Task Tutorial

Key Features:

- Use loops, arrays, and conditional statements for game logic.
- Display the game board dynamically

D. INTERNSHIP TASK RECORDS

The following task records document the project-based work completed during the internship.

1

Due Date: 19 Aug 2026

Submitted on track

Student Management System

To develop a console-based Student Management System in C++ that efficiently manages student records using file handling and menu-driven operations.

Watch Task Tutorial

Key Features:

- Add, update, delete, and display student information.
- Use file handling to store data persistently.
- Implement menu-driven operations for smooth navigation.

Expected Outcome:

A functional application that allows users to add, update, delete, and display student information with persistent data storage for reliable record management.

2

Due Date: 26 Aug 2026

Submitted on track

Bank Management Application

To design and implement a C++ based Bank Management Application that simulates core banking operations using object-oriented programming and file handling.

Watch Task Tutorial

Key Features:

- Include features like deposit, withdrawal, and balance check.
- Use object-oriented programming concepts.
- Store customer data securely using file management.

Expected Outcome:

A secure and functional banking system capable of performing deposits, withdrawals, and balance inquiries while maintaining persistent customer records.

3

Due Date: 02 Sept 2026

Submitted on track

Library Management

To develop a Library Management Application that efficiently manages book and borrowing records using object-oriented programming.

Watch Task Tutorial

Key Features:

- Implement structures or classes for book and member details.
- Support book issue and return.
- Include search functionality by author.

Expected Outcome:

A functional application that supports adding, issuing and returning books, searching by title or author, and managing library operations.

Completed Successfully

39

18

Certificate

O F A C H I E V E M E N T

Sanjay.G

This is to certify that **Sanjay.G** has successfully completed an internship in
C++ Programming from **13 Aug 2026** to **12 Sept 2026**.



VERIFIED CERTIFICATE
ID: THX-AUG1326-205



Hariharan M
Founder & CEO

END OF REPORT

Sanjay G

B.TECH Artificial Intelligence & Data Science

J.N.N Institute of Engineering | Academic Year 2026–2027